Taller no paramétrico

Riobamba, Ecuador

Jeffrey S. Racine
Universidad McMaster

miércoles, 19 de abril de 2023

Consejos profesionales de diapositivas

- Enlace a las diapositivas: jeffreyracine.github.io/RBworkshop (distingue entre mayúsculas y minúsculas) (Traductor de Google)
- Ver **pantalla completa** presionando la tecla F (presione la tecla Esc para revertir)
- Acceda **al menú de navegación** presionando la tecla M (menú de navegación X para cerrar)
- **Avanzar** usando las teclas de flecha
- **Haga zoom** manteniendo presionada la tecla Alt en Windows, la tecla Opt en macOS o la tecla Ctrl en Linux y haciendo clic en cualquier elemento de la pantalla (Alt/Opt/Ctrl haga clic de nuevo para alejar)
- Utilice el botón **Copiar al portapapeles** para bloques de código R (parte superior derecha del bloque) para copiar y pegar en R/RStudio
- **Exporte a un PDF** presionando la tecla E (espere unos segundos, luego imprima [o imprima

¡Bienvenido!

- El sitio web con información de instalación para R, RStudio y$\TeX$ es https://jeffreyracine.github.io/RBworkshop
- El repositorio de GitHub con código de ejemplo es https://github.com/JeffreyRacine/RBworkshop
- No seas tímido, siéntete libre de hacer preguntas
- ¡Vámonos!

¡Bienvenido!

- Se le guiará a través de métodos kernel no paramétricos básicos usando R
- También, si es paciente, se le presentarán herramientas lanzadas recientemente para realizar investigaciones reproducibles.
- No se requieren conocimientos previos de análisis no paramétrico ni del lenguaje de programación R
- Todo el software está disponible de forma *gratuita* y es *de código abierto.*
- Para obtener descripciones más detalladas, consulte Racine (2019) , Racine (2012) y Hayfield y Racine (2008).

Descripción general

- Espero que deje este taller armado con un conjunto moderno de herramientas de análisis de datos.
- Analicemos brevemente lo siguiente:
  - ¿Qué es R?
  - ¿Qué es RStudio?
  - ¿Qué es la regresión del núcleo?
  - ¿Qué es Cuarto?

Tipos de datos de fondo

R y tipos de datos

> **Importante**
>
> Datos *prefabricados antes del análisis* para que las funciones R puedan hacer su trabajo

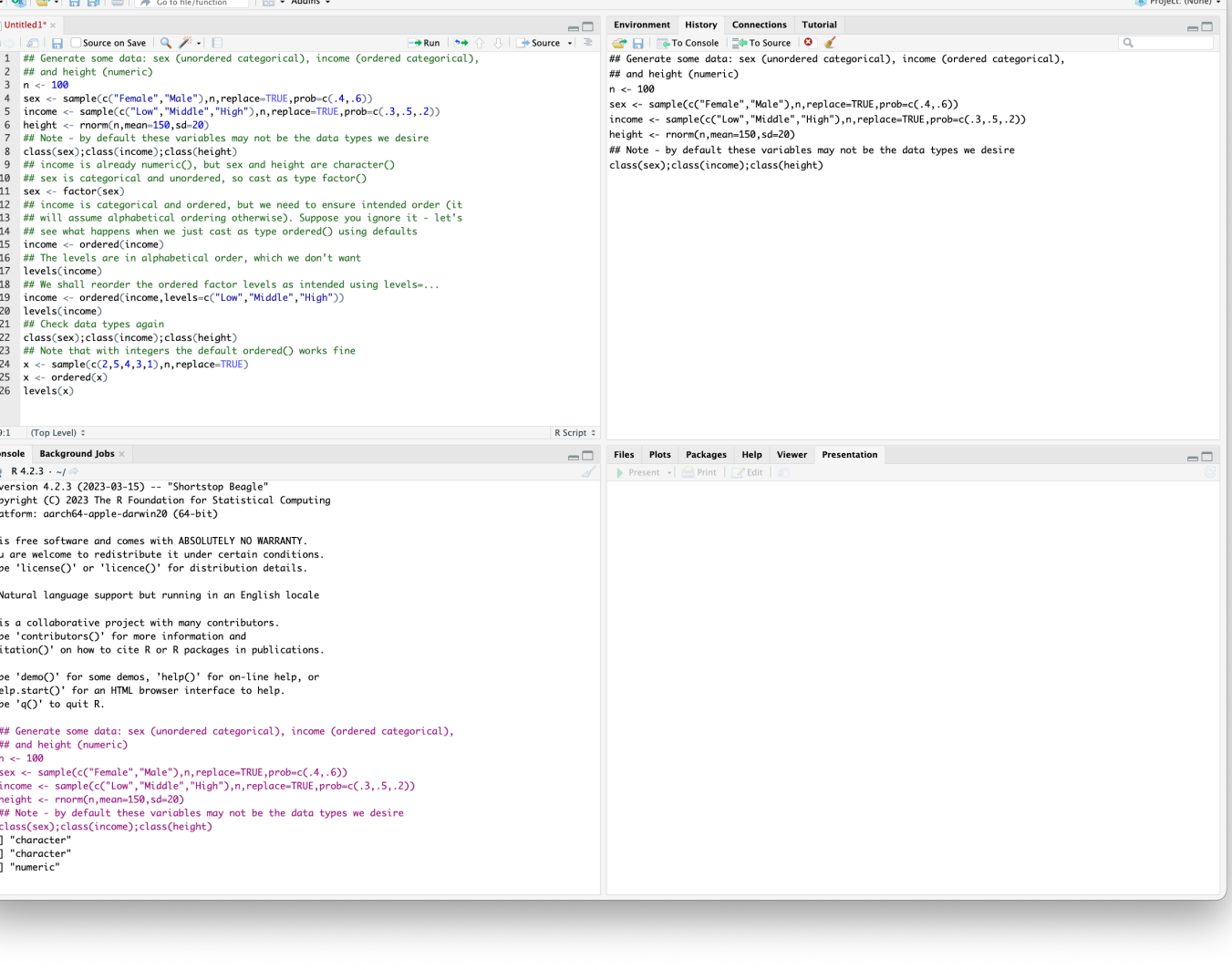
- Es *crucial* comprender los diferentes tipos de datos en R
- Hay tres funciones de R que usaremos: `numeric()`, `factor()`y`ordered()`
- Estos corresponden a datos que son *números* , *categorías desordenadas* y *categorías ordenadas* , respectivamente.
- Presta mucha atención al siguiente ejemplo

R y tipos de datos

- A continuación se muestra un ejemplo de cómo proyectamos *nuestros* datos en R *antes* de analizarlos.
- Copie y pegue esto en un nuevo archivo R en RStudio, luego ejecútelo línea por línea

```r
## Generate some data: sex (unordered categorical), income (ordered categorical),
## and height (numeric)
n <- 100
sex <- sample(c("Female","Male"),n,replace=TRUE,prob=c(.4,.6))
income <- sample(c("Low","Middle","High"),n,replace=TRUE,prob=c(.3,.5,.2))
height <- rnorm(n,mean=150,sd=20)
## Note - by default these variables may not be the data types we desire
class(sex);class(income);class(height)
## income is already numeric(), but sex and height are character()
## sex is categorical and unordered, so cast as type factor()
sex <- factor(sex)
## income is categorical and ordered, but we need to ensure intended order (it
## will assume alphabetical ordering otherwise). Suppose you ignore it - let's
## see what happens when we just cast as type ordered() using defaults
income <- ordered(income)
## The levels are in alphabetical order, which we don't want
levels(income)
## We shall reorder the ordered factor levels as intended using levels=...
income <- ordered(income,levels=c("Low","Middle","High"))
levels(income)
## Check data types again
class(sex);class(income);class(height)
## Note that with integers the default ordered() works fine
x <- sample(c(2,5,4,3,1),n,replace=TRUE)
x <- ordered(x)
levels(x)
```

Interfaz del editor RStudio R



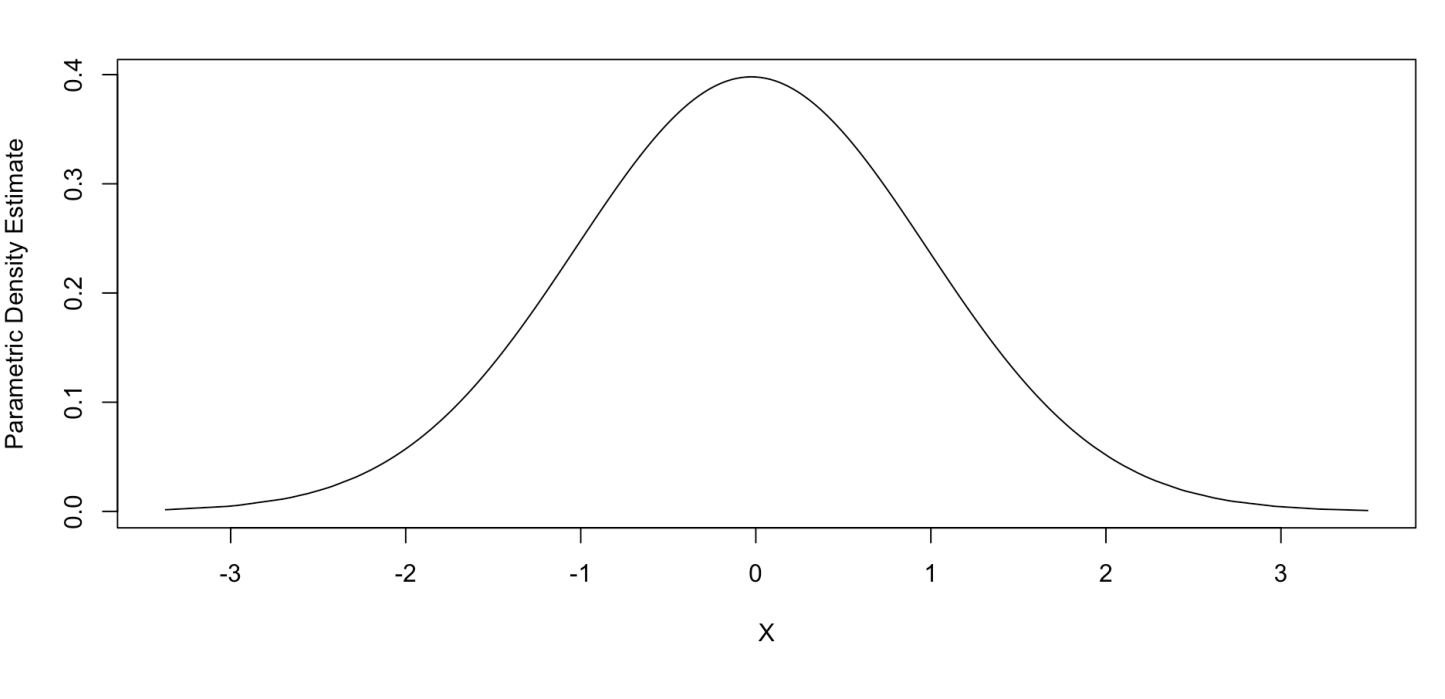
```
R 4.2.3 · ~/
R version 4.2.3 (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: aarch64-apple-darwin20 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> ## Generate some data: sex (unordered categorical), income (ordered categorical),
> ## and height (numeric)
> n <- 100
> sex <- sample(c("Female","Male"),n,replace=TRUE,prob=c(.4,.6))
> income <- sample(c("Low","Middle","High"),n,replace=TRUE,prob=c(.3,.5,.2))
> height <- rnorm(n,mean=150,sd=20)
> ## Note - by default these variables may not be the data types we desire
> class(sex);class(income);class(height)
[1] "character"
[1] "character"
[1] "numeric"
>
```

Estimación de densidad de fondo

¿Paramétrico o no paramétrico?

Supongamos que necesitamos estimar la densidad$f(x)$de alguna variable aleatoria numérica$X$

```r
## Let's simulate a random numeric sample from the normal distribution
set.seed(42)
n <- 1000
x <- rnorm(n)
## Let's sort the data so we can graph x versus dnorm(x,...) using lines (type="l")
x <- sort(x)
## Since we simulated the data, let's plot the true, known, parametric density
## (we can't do this with actual data because the density of such data is, in
## general, unknown)
plot(x,dnorm(x,mean=mean(x),sd=sd(x)),type="l",ylab="Parametric Density Estimate",xlab="X")
```



¿Paramétrico o no paramétrico?

- La función R `shapiro.test(x)`prueba la normalidad

Prueba de normalidad de Shapiro-Wilk:`x`

Estadística de prueba	valor p
0.9988	0.767

- Estos son datos *normales simulados* , y no podemos rechazar el valor nulo

> **Advertencia**
>
> ¿Qué pasa si nos equivocamos en la densidad nula (es decir, qué pasa si rechazamos la

¿Paramétrico o no paramétrico?

- Ahora vamos a probar la normalidad de los datos reales,`eruptions`

Prueba de normalidad de Shapiro-Wilk:`eruptions`

Estadística de prueba	valor p
0.8459	9.036e-16 * * *

> **Advertencia**
>
> Oh oh... Supongo que descartamos el modelo paramétrico normal... ¿Qué sigue?

¿Paramétrico o no paramétrico?

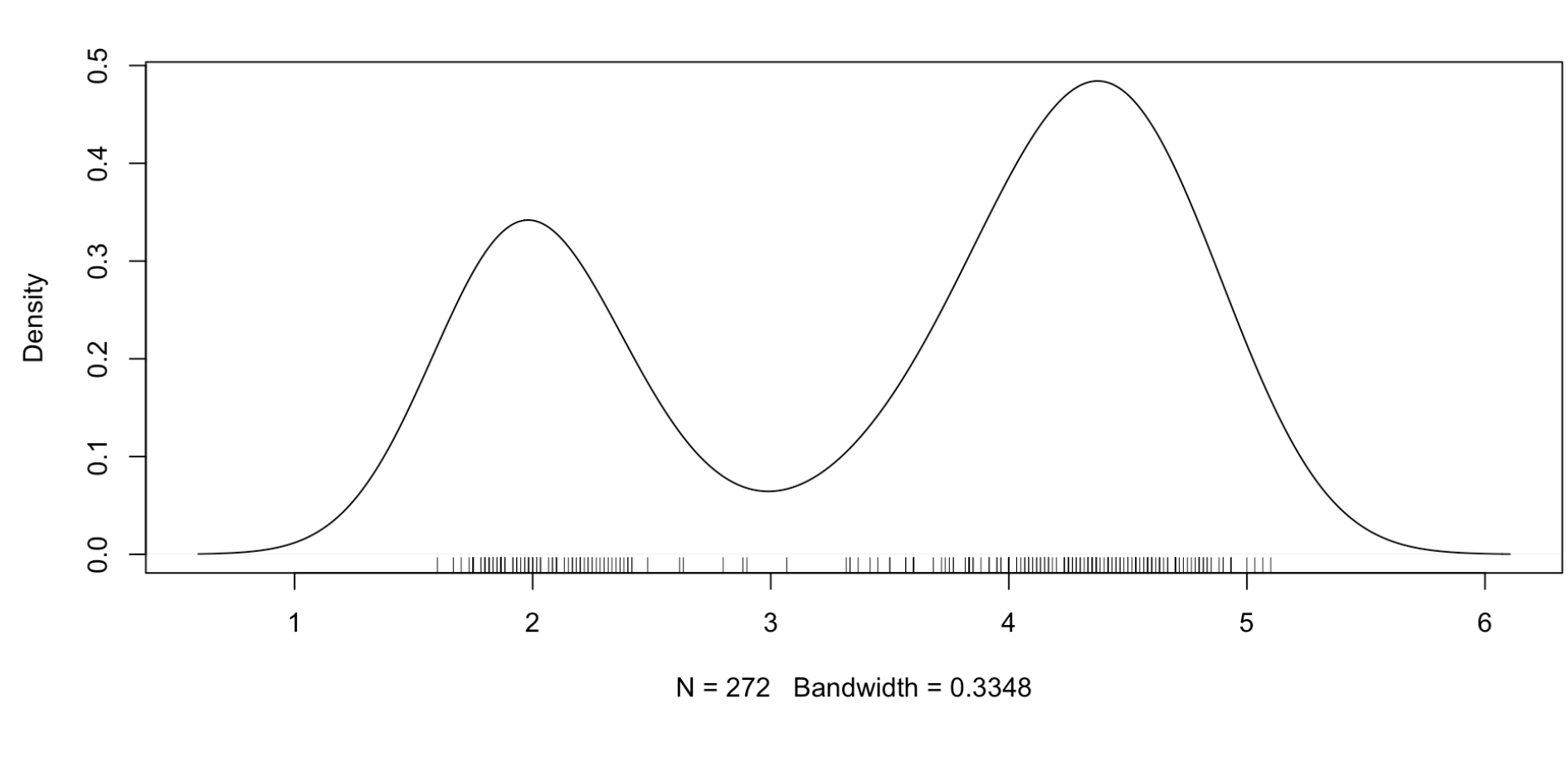
- Información actual (X_i,$i = 1, \ldots, n$) rara vez se extrae de las distribuciones paramétricas simples utilizadas en *la teoría del muestreo*
- La teoría del muestreo describe *estadísticas resumidas* , por ejemplo, *promedios* de datos reales o *promedios* de desviaciones al cuadrado, como

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} X_i \quad \text{or} \quad \hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu})^2$$

- Pero necesitamos estimar la *densidad de los datos reales* (es decir, $f(x)$), no la *densidad de alguna estadística de resumen* (es decir,

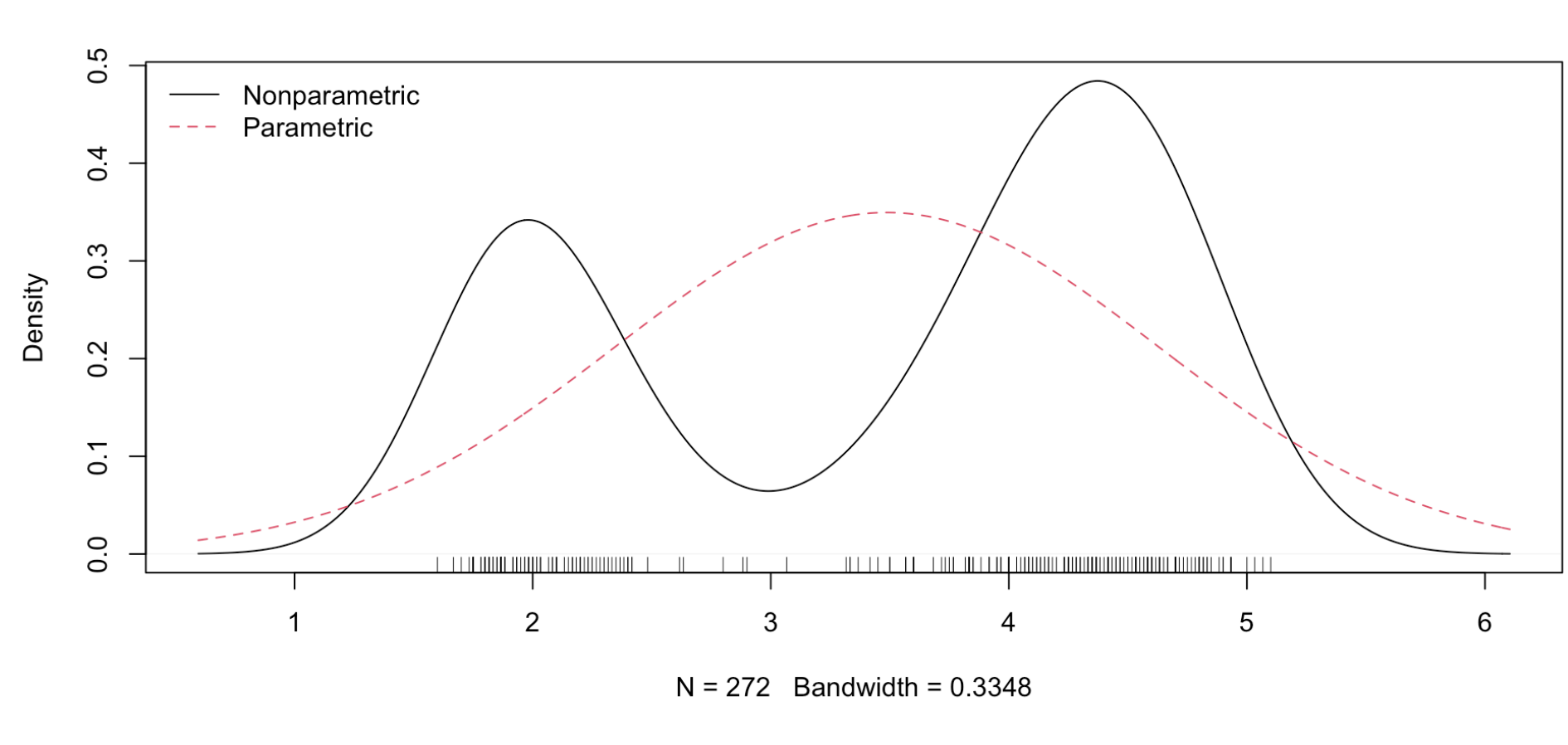
¿Paramétrico o no paramétrico?

- A continuación se muestra una estimación de densidad no paramétrica y *una alfombra* de datos para los `eruptions`datos (es bimodal *y* asimétrica)



¿Paramétrico o no paramétrico?

- *A continuación se muestra una estimación paramétrica normal* y no paramétrica (recuerde que el modelo paramétrico normal ha sido *rechazado*)



¿Paramétrico o no paramétrico?

- Aquí radica el quid del problema paramétrico.
- Anotamos algún modelo paramétrico que se extrae de un espacio *denso* de funciones (su número es *incontable*)
- Una suposición muy importante es que *este modelo paramétrico es el modelo verdadero* , y todas las propiedades del modelo (imparcialidad, consistencia, etc.) dependen *crucialmente* de que este sea el caso.

- Pero si habla en serio, *pruebe* inmediatamente su modelo para ver si tiene las especificaciones correctas.
- ¿Qué sucede si se rechaza el modelo paramétrico, como sucede a menudo?

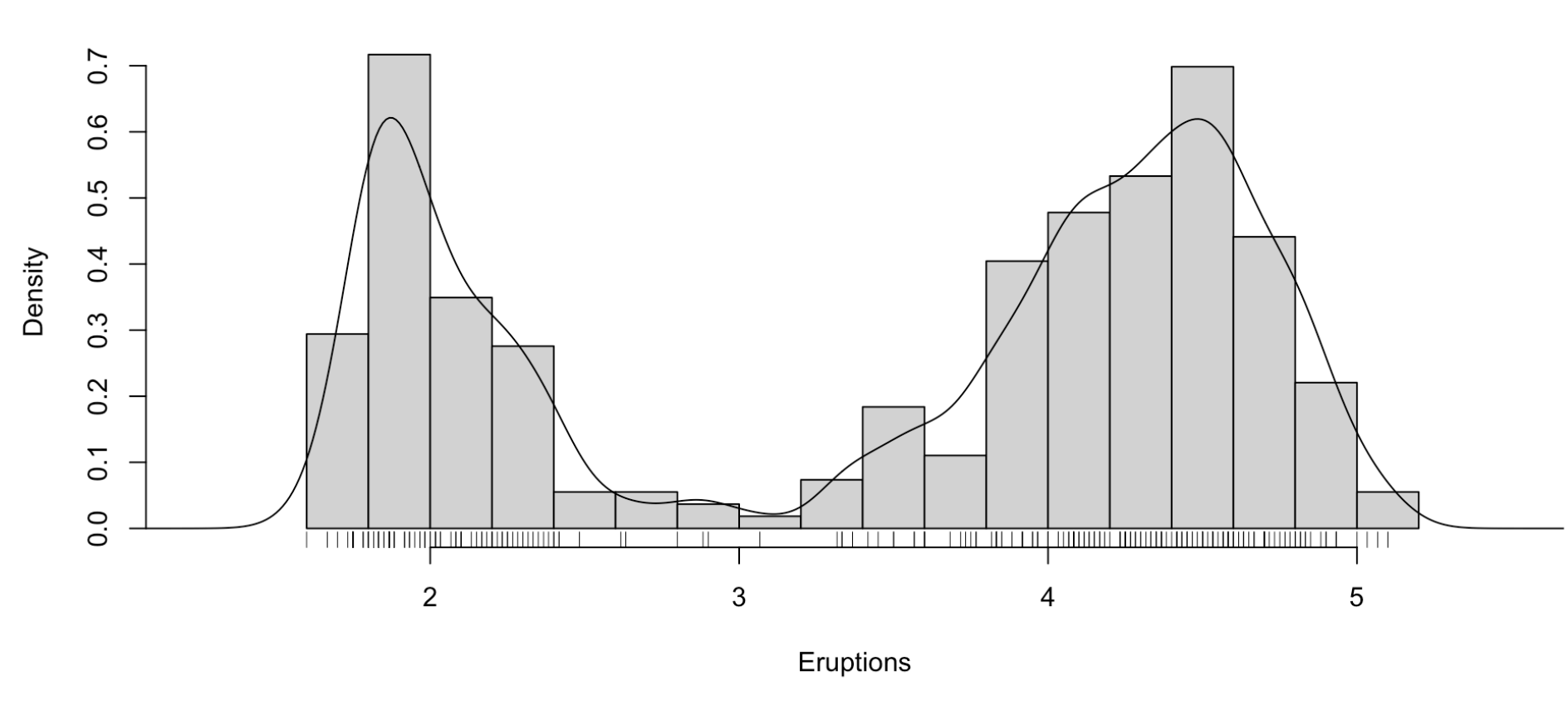
Datos numéricos esenciales no paramétricos

Datos numéricos

- Los métodos paramétricos requieren que el usuario haga *suposiciones muy fuertes* sobre el *proceso de generación de datos* (DGP)
- Acabamos de considerar la estimación de la densidad, pero exactamente el mismo problema afecta a todos los análisis paramétricos (es decir, regresión, etc.)
- Consideraremos, en cambio, estimadores *kernel no paramétricos*
- Un *núcleo* es simplemente una función de peso, que para datos *numéricos* denotamos por$K(Z_i)$dónde$Z_i = (x - X_i)/h$
- Damos *mayor* peso a las observaciones cercanas ax(es decir, pequeñoZ_i) que a los que yacen más lejos (es decir, grandesZ_i)

¿No liso o liso?

- A continuación se muestran estimaciones de densidad no paramétrica suave y no suave para la variable *numérica*`eruptions`



¿No liso o liso?

- El estimador de densidad de histograma no paramétrico no suave es

$$f_H(x) = \frac{1}{nh}\sum_{i=1}^{n} \mathbf{1}(X_i \text{ is in the same bin as } x)$$

- Este estimador tiene inconvenientes, que incluyen
  - no es particularmente *eficiente* en el sentido estadístico
  - la discontinuidad del estimador presenta obstáculos si se requieren derivados ($df_H(x)/dx$es 0 o indefinido)

Estimación suave de densidad univariante

- Considere una función kernel suave simétrica que satisface $K(z) \geq 0$y$\int_{-\infty}^{\infty} K(z)\,dz = 1$
- El estimador de densidad kernel no paramétrico suave es

$$\hat{f}(x) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right)$$

- Este estimador es dominante en entornos aplicados, aunque *el sesgo de límite* puede ser un problema (existen correcciones simples, vea `npuniden.boundary()` en el `np`paquete)

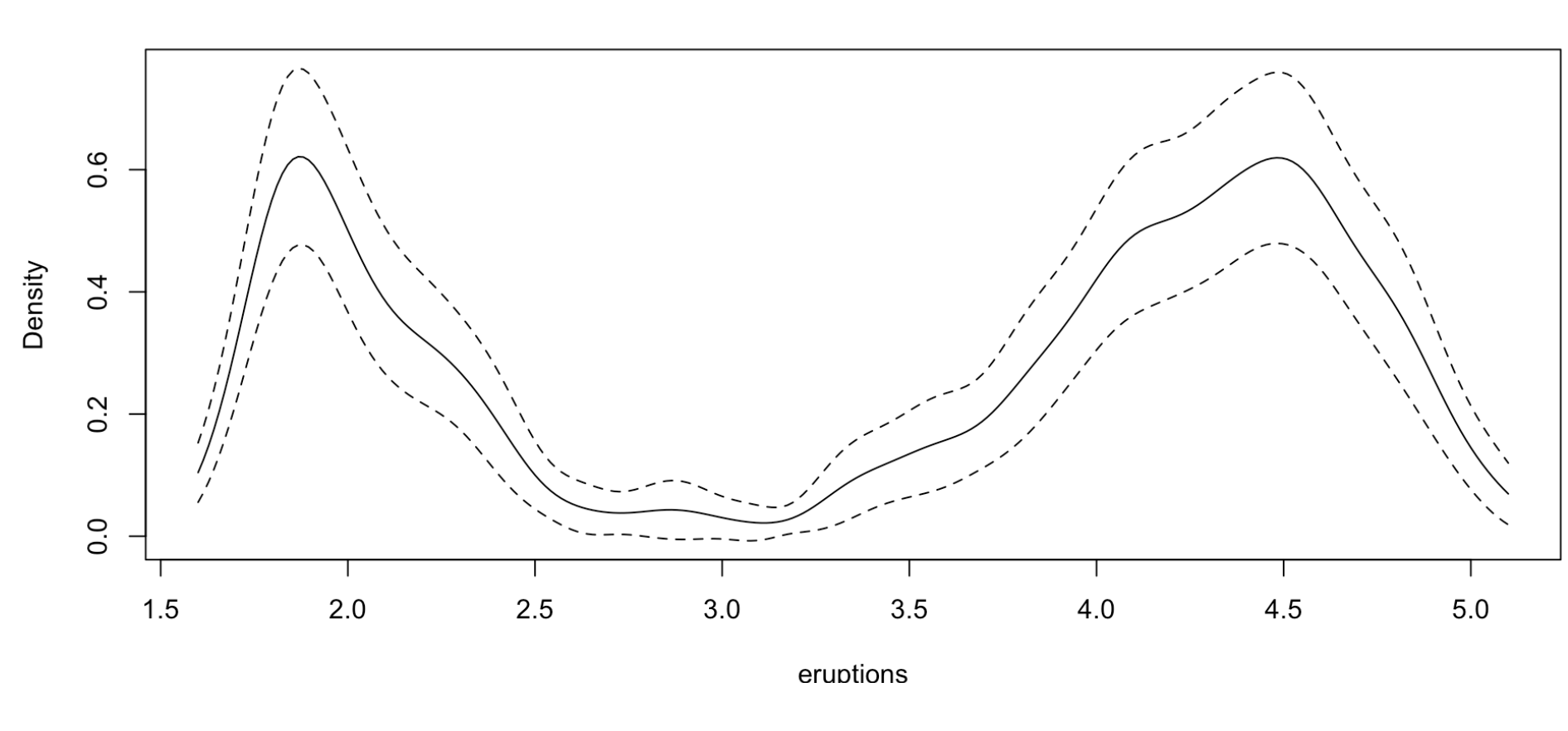
Estimación suave de densidad univariante

> **Importante**
>
> - La función del núcleo$K(z)$es relativamente poco importante (imparte *suavidad* en la estimación, donde las funciones suaves son funciones *continuamente diferenciables*)
> - el ancho de banda$h$es crucial (gobierna la compensación sesgo-varianza)

- *La cercanía* está determinada por un *ancho de banda* , denotado$h$
- Nosotros elegimos$h$para minimizar *el riesgo de error cuadrático* y compensar *el sesgo* por *la varianza* de la muestra en cuestión
- Para lograr esto, utilizamos métodos *basados en datos* para seleccionar$h$(por ejemplo, validación cruzada de mínimos

Estimación suave de densidad univariante

```r
library(np)
data(faithful)
fhat <- npudens(~eruptions,data=faithful)
plot(fhat,neval=250,plot.errors.method="bootstrap")
```



Estimación de la densidad de juntas lisas

- Hasta ahora hemos considerado densidades univariadas ($X \in \mathbb{R}^1$)
- Dejar$X \in \mathbb{R}^q$denota un vector numérico de dimensiónq
- Dejar$f(x) = f(x_1, x_2, \dots, x_q)$denota una PDF conjunta evaluada en$x = (x_1, x_2, \dots, x_q)'$
- Dejar$\{X_1, X_2, \dots, X_n\}$representar$n$sorteos de un vector aleatorio numérico$X$, donde el$i$el sorteo se denota por $X_i = (X_{i1}, X_{i2}, \dots, X_{iq})$

Estimación de la densidad de juntas lisas

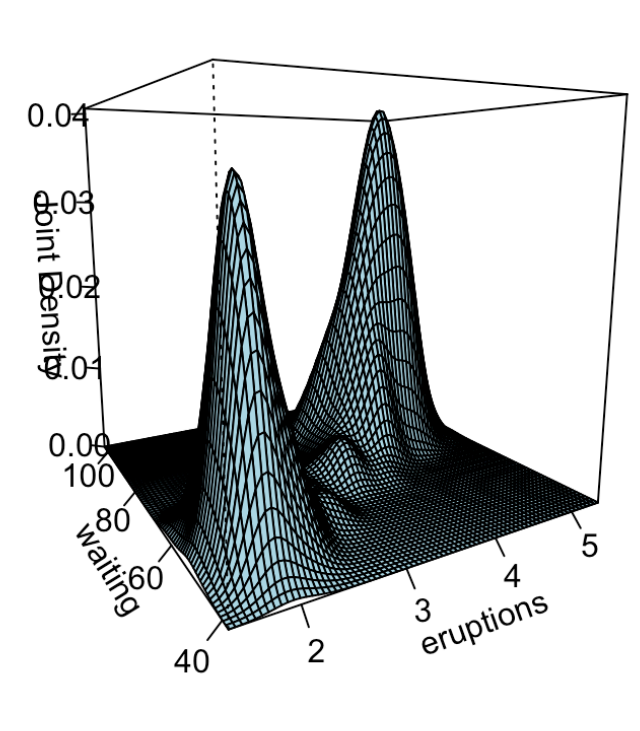
- El estimador de densidad kernel multivariado es

$$\hat{f}(x) = \frac{1}{nh_1 \dots h_q}\sum_{i=1}^{n} K\left(\frac{x_1 - X_{i1}}{h_1}, \dots, \frac{x_q - X_{iq}}{h_q}\right)$$

- $K(\cdot)$es un kernel *multivariable* (típicamente el *producto* de kernels univariantes - tenemos mucha flexibilidad aquí)
- Seguimos utilizando métodos basados en datos (por lo general, validación cruzada) para la selección de ancho de banda cuando $q \geq 1$(los anchos de banda óptimos difieren según las variables, es decir,$h_1 \neq h_2$, etc.)

Estimación de la densidad de juntas lisas

```r
library(np)
data(faithful)
fhat <- npudens(~eruptions+waiting,data=faithful)
plot(fhat,theta=330,xtrim=-0.05,neval=75,view="fixed",main="")
```



Datos categóricos esenciales no paramétricos

Variables categóricas

- Consideramos la densidad de variables aleatorias distribuidas continuamente$X \in \mathbb{R}$, pero también nos ocupamos de la función de probabilidad de las variables categóricas$X \in \mathcal{D}$ dónde$\mathcal{D}$es un conjunto discreto
- Podríamos suponer un modelo paramétrico para la función de probabilidad de$X \in \mathcal{D}$pero nos enfrentamos exactamente al mismo problema que enfrentamos antes
- La contraparte categórica no uniforme del histograma se denomina estimador de *frecuencia* o probabilidad *empírica* y, al igual que el histograma, tiene algunos inconvenientes.

Estimación de probabilidad univariada

- Dejar$X \in \mathcal{D} = \{0, 1, \dots, c-1\}$ser categórico
- Xpuede ser *desordenado* (`factor()` en R) u *ordenado* (`ordered()` en R)
- El estimador *de frecuencia* no paramétrico de$p(x)$es

$$\begin{aligned} p_n(x) &= \frac{\#X_i \text{ equal to } x}{n} \\ &= \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(X_i = x) \end{aligned}$$

Estimación de probabilidad univariada

- El estimador kernel no paramétrico *desordenado de*$p(x)$es

$$\hat{p}(x) = \frac{1}{n}\sum_{i=1}^{n} L(X_i, x, \lambda)$$

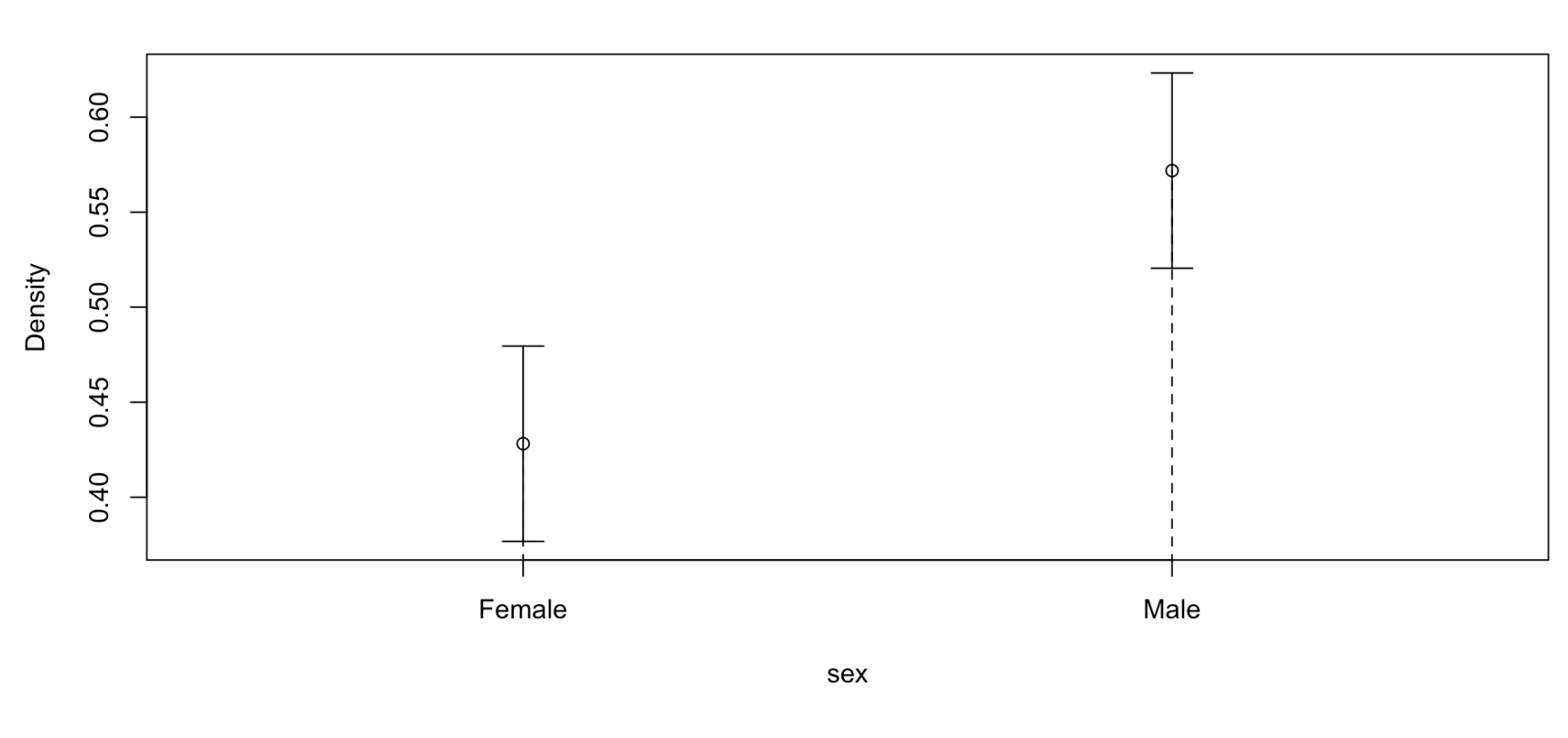
- $L(\cdot)$es una función del kernel *desordenada* dada por

$$L(X_i, x, \lambda) = \begin{cases} 1 - \lambda & \text{if } X_i = x \\ \lambda/(c-1) & \text{otherwise} \end{cases}$$

- λes un *parámetro de suavizado* (contraparte del *ancho de banda* h)

Ejemplo - Probabilidad desordenada

```r
n <- 250
set.seed(42)
sex <- sample(c("Female","Male"),n,replace=TRUE,prob=c(.4,.6))
sex <- factor(sex)
phat <- npudens(~sex)
plot(phat,plot.errors.method="bootstrap")
```



Estimación de probabilidad univariada

- Dejar$X \in \mathcal{D} = \{0, 1, \dots, c-1\}$,$c \geq 2$, ser *ordenado*
- El estimador kernel no paramétrico *ordenado de*$p(x)$es

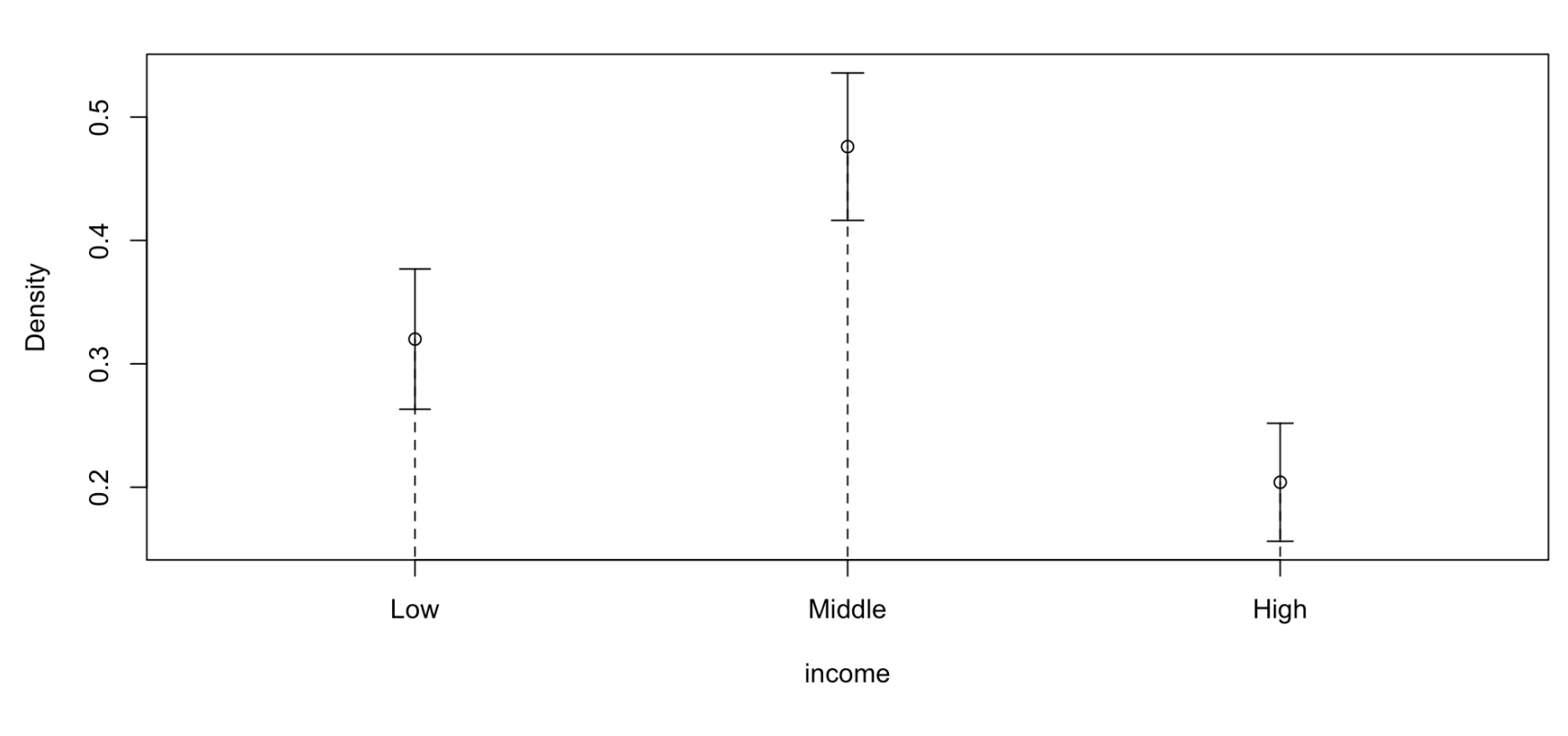
$$\hat{p}(x) = \frac{1}{n}\sum_{i=1}^{n} l(X_i, x, \lambda)$$

- $l(X_i, x, \lambda)$es una función kernel *ordenada* dada por

$$l(X_i, x, \lambda) = \frac{\lambda^{d_{xi}}}{\Lambda_i}$$

Ejemplo: probabilidad ordenada

```r
n <- 250
set.seed(42)
income <- sample(c("Low","Middle","High"),n,replace=TRUE,prob=c(.3,.5,.2))
income <- ordered(income,levels=c("Low","Middle","High"))
phat <- npudens(~income)
plot(phat,plot.errors.method="bootstrap")
```



Datos mixtos esenciales no paramétricos

Estimación de densidad de datos mixtos

- Los estadísticos saben que $f(x)$y$p(x)$ambas se denominan funciones *de densidad* (la diferencia radica en su *medida* ; la última utiliza una medida *de conteo*), por lo que adoptamos $f(x)$
- Suponga que tiene una densidad conjunta definida sobre tipos de datos mixtos, digamos, un numérico ($X^c \in \mathbb{R}$) y uno desordenado ($X^d \in \mathcal{D}$con cardinalidad$c$)
- Nos gustaría modelar su función de densidad conjunta $f(x) = f(x^c, x^d)$, donde los superíndices^{c}y^{d}denotan tipos de datos continuos y discretos, respectivamente, y donde $x = (x^c, x^d) \in \mathbb{R} \times \mathcal{D}$

Estimación de densidad de datos mixtos

- El estimador kernel de$f(x^c, x^d)$es

$$\hat{f}(x^c, x^d) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{h} K\left(\frac{X_i^c - x^c}{h}\right) L(X_i^d, x^d, \lambda)$$

- $K(\cdot)$y$L(\cdot)$son funciones kernel (univariadas) *numéricas* y *desordenadas , respectivamente*
- Si tuviera una variable numérica y una *ordenada* , usaría el núcleo ordenado$l(X_i^d, x^d, \lambda)$arriba
- Ahora consideremos el caso *general* de densidad de datos mixtos multivariados

Estimación de densidad de datos mixtos

- En entornos multivariantes generales, la función de densidad de probabilidad$f(x)$podría usar alguna combinación de$q$numérico, $r$desordenado, y$s$tipos de variables ordenadas (xy$X_i$son $q + r + s$-vectores)
- El kernel (producto) para estimar la función de densidad conjunta es

$$\prod_{j=1}^{q} h_j^{-1} K\left(\frac{x_j^c - X_{ij}^c}{h_j}\right) \prod_{j=1}^{r} L(X_{ij}^u, x_j^u, \lambda_j^u) \prod_{j=1}^{s} l(X_{ij}^o, x_j^o, \lambda_j^o)$$

- Todo lo que estamos haciendo aquí es usar la *función de kernel adecuada* para cada tipo de datos.

Estimación de densidad de datos mixtos

- Dejar$\gamma = (h_1, \dots, h_q, \lambda_1^u, \dots, \lambda_r^u, \lambda_1^o, \dots, \lambda_s^o)$
- Llame a la función kernel del producto en la diapositiva anterior el kernel *generalizado* , es decir, deje que la expresión se escriba como

$$K_\gamma(X_i, x) = \prod_{j=1}^{q} [\dots] \prod_{j=1}^{r} [\dots] \prod_{j=1}^{s} [\dots]$$

- Con$x$y$X_i$ser$q + r + s$vectores, escribimos$\hat{f}(x)$como

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} K_\gamma(X_i, x)$$

Estimación de densidad de datos mixtos

- El punto clave es que *una vez que envía sus datos* , las funciones R en el `np`paquete saben *exactamente* qué hacer
- Usan *automáticamente* el kernel apropiado para el tipo de datos apropiado (numérico, factorial, ordenado)
- Los métodos de ancho de banda basados en datos se ajustan automáticamente al tipo de datos
- Los métodos de inferencia (intervalos de confianza, pruebas de significancia en regresión) hacen lo mismo
- Consideremos un ejemplo rápido

Ejemplo: densidad de datos mixta

- Usamos `wage1`los datos de Wooldridge y consideramos dos variables, una numérica (`lwage`) y otra ordenada (`numdep`)
- `lwage`es el logaritmo de los ingresos promedio por hora de un individuo y `numdep`es su número de dependientes
- El número de observaciones en cada *celda* se tabula en la Tabla 1

Tabla 1: Recuento del número de dependientes presentes en 526 hogares por celda

número dep	0	1	2	3	4	5	6
cuenta	252	105	99	45	dieciséis	7	2

- Estimamos$\hat{f}(lwage, numdep)$y trazarlo en la figura 1

Ejemplo: densidad de datos mixta

- A continuación se muestra el código R para estimar y trazar la densidad conjunta (usamos el `plot3D`paquete y reformateamos los datos para representar este gráfico)

```r
library(np)
library(plot3D)
data(wage1)
bw <- npudensbw(~lwage+ordered(numdep),data=wage1)
numdep.seq <- with(wage1,sort(unique(numdep)))
lwage.seq <- with(wage1,seq(min(lwage),max(lwage),length=50))
wage1.eval <- expand.grid(numdep=ordered(numdep.seq),lwage=lwage.seq)
fhat <- fitted(npudens(bws=bw,newdata=wage1.eval))
## Hack since scatter3D converts ordered 0-6 to numeric 1-7
scatter3D(as.numeric(wage1.eval[,1])-1,wage1.eval[,2],fhat,
          ylab="Log-Wage",
          xlab="Number of Dependants",
          zlab="Joint Density",
          ticktype="detailed",
          angle=15,
          box=TRUE,
          type="h",
          grid=TRUE,
          col="blue",
          colkey=FALSE)
```

Ejemplo: densidad de datos mixta

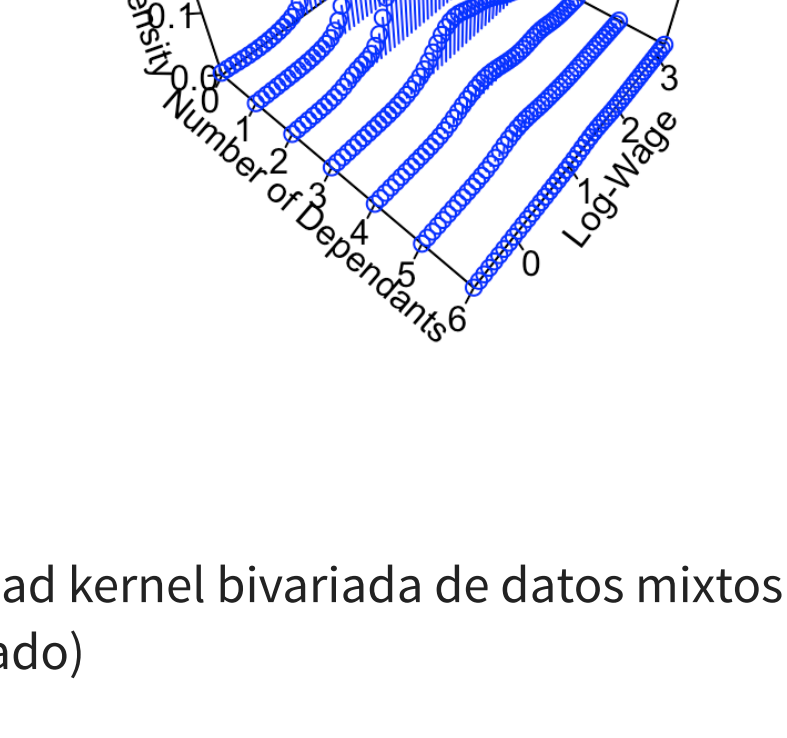


Figura 1: Estimación de densidad kernel bivariada de datos mixtos para la PDF conjunta de lwage (numérico) y numdeps (ordenado)

Regresión no paramétrica

Regresión no paramétrica

- La función media condicional$g(x) := \mathbb{E}(Y|X = x)$para numérico$Y \in \mathbb{R}$se *define* como

$$g(x) = \int y \frac{f(y,x)}{f(x)}\, dy = \frac{m(x)}{f(x)} \qquad (1)$$

- Empecemos con$X \in \mathbb{R}^1$(es decir, un predictor numérico)
- Nuestro objetivo es estimar el modelo de regresión desconocido $y = g(x) + \varepsilon$sin suponer que, por ejemplo,$g(x) = \beta_0 + \beta_1 x$

Regresión no paramétrica

- La regresión es *difícil* porque$f(y,x)$y$f(x)$son desconocidos
- Se pueden estimar *consistentemente* usando$\hat{f}(y,x)$y$\hat{f}(x)$
- Para distribución continua$Y \in \mathbb{R}$, el estimador de regresión kernel *constante local* reemplaza el *desconocido* $f(y,x)$y$f(x)$con *known* $\hat{f}(y,x)$y$\hat{f}(x)$en(1), por eso

$$\hat{g}(x) = \int y \frac{\hat{f}(y,x)}{\hat{f}(x)}\, dy = \frac{\hat{m}(x)}{\hat{f}(x)} \qquad (2)$$

Regresión no paramétrica

- Para$X \in \mathbb{R}^1$, estimamos$g(x)$reemplazando lo desconocido$f(x)$ en(1)con

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right),$$

- y reemplazando lo desconocido$f(y,x)$en(1)con

$$\hat{f}(y,x) = \frac{1}{nh_y h} \sum_{i=1}^{n} K\left(\frac{y - Y_i}{h_y}\right) K\left(\frac{x - X_i}{h}\right)$$

Regresión no paramétrica

- Algunas simplificaciones matemáticas de$\hat{g}(x)$lleva a

$$\hat{g}(x) = \sum_{i=1}^{n} Y_i \left\{ \frac{K\left(\frac{X_i - x}{h}\right)}{\sum_{i=1}^{n} K\left(\frac{X_i - x}{h}\right)} \right\} = \sum_{i=1}^{n} Y_i W_i(x)$$

- Con datos mixtos multivariados$X$(es decir,xy$X_i$son$q + r + s$-vectores) reemplazamos$K(\cdot)$con$K_\gamma(X_i, x)$
- La función de regresión no paramétrica estimada es

$$y = \hat{g}(x) + \hat{\epsilon} \qquad (3)$$

Ilustración de datos simulados

```r
library(np)
set.seed(42)
n <- 1000
x <- sort(runif(n))
dgp <- cos(2*pi*x)
y <- dgp + rnorm(n,sd=0.25*sd(dgp))
ghat <- npreg(y~x)
plot(x,y,cex=0.5,col="grey",xlab="X",ylab="Y")
lines(x,dgp)
lines(x,fitted(ghat),col=2)
abline(ghat.ols <- lm(y~x),col=3)
legend("top",c("DGP","Kernel","OLS"),col=1:3,lty=1,bty="n")
```

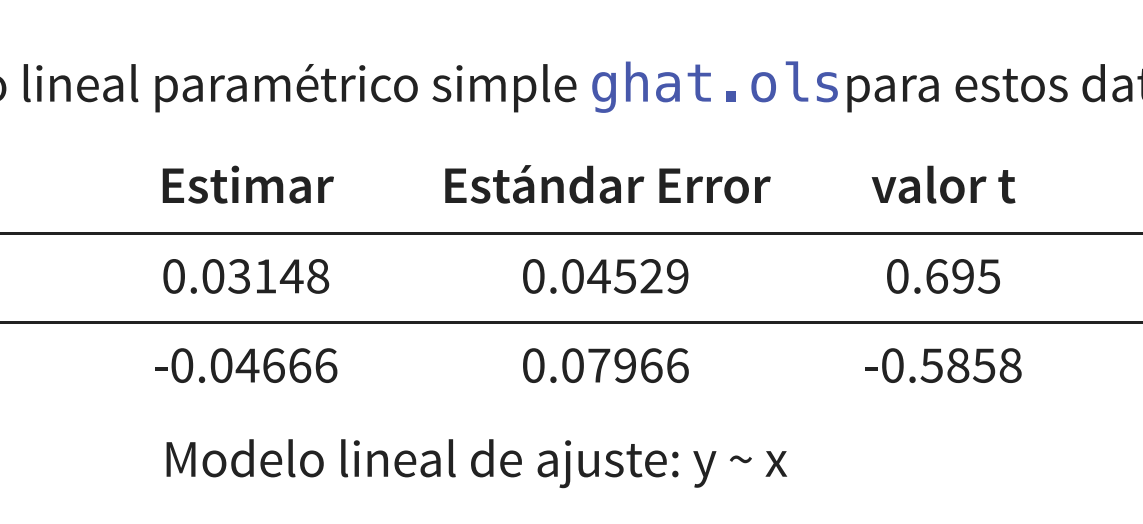


Ilustración de datos simulados: OLS

¿ Cómo funciona el modelo lineal paramétrico simple `ghat.ols`para estos datos simulados?

	Estimar	Estándar Error	valor t	Pr(>|t|)
(Interceptar)	0.03148	0.04529	0.695	0.4872
X	-0.04666	0.07966	-0.5858	0.5582

Modelo lineal de ajuste: y ~ x

Observaciones	estándar residual Error	R^2	EquilibradoR^2
1000	0.7338	0.0003437	-0.000658

> **Advertencia**
>
> Oh oh... este modelo paramétrico lineal realmente apesta... el modelo no paramétrico tiene unR^2de *0,9411* (el modelo paramétrico tiene unR^2de *0.0003437* y un valor ajustado *negativo*R^2)

Efectos marginales

- el univariado$X$ *La función de efectos marginales* es simplemente la función de la primera derivada parcial y se define como

$$\beta(x) = \frac{dg(x)}{dx} = \frac{f(x)m'(x) - m(x)f'(x)}{f^2(x)}$$
$$= \frac{m'(x)}{f(x)} - g(x)\frac{f'(x)}{f(x)} \qquad (4)$$

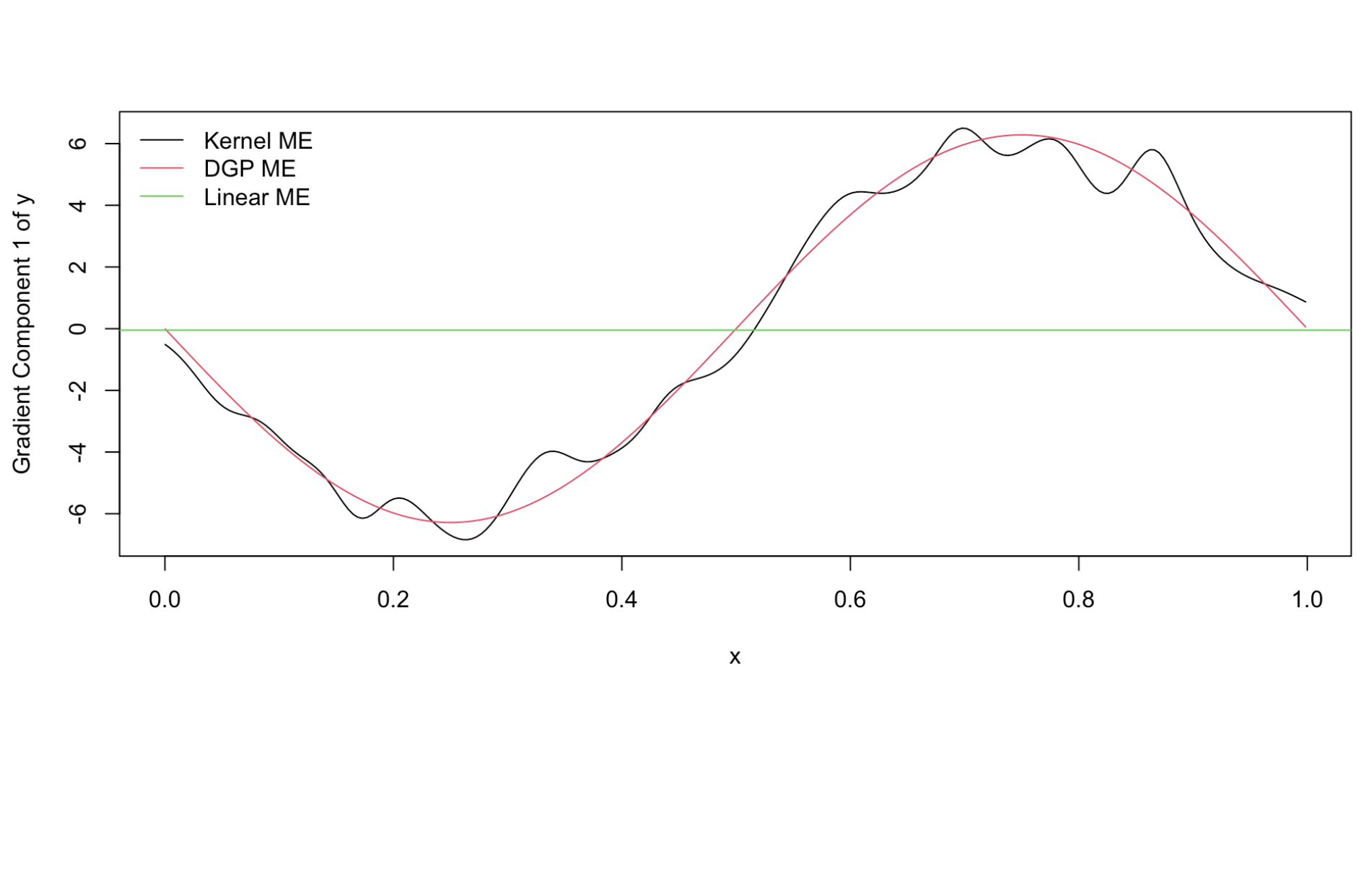
- Nosotros construimos$\hat{\beta}(x)$por reemplazo$f(x)$,$m'(x)$,$m(x)$, y $f'(x)$en(4)con$\hat{f}(x)$,$\hat{m}'(x)$,$\hat{m}(x)$, y$\hat{f}'(x)$

Efectos marginales

- Recuerde las definiciones de las funciones media condicional y de efectos marginales,$g(x)$y$\beta(x)$
- Implican densidades conjuntas y marginales desconocidas y sus derivadas, que son funciones de$x$yy
- Si tuviera la tarea de estimar$\beta(x)$, es difícil justificar la suposición de que$\beta(x)$es *constante* (es decir, alguna constanteβeso *no* es funcion de$x$como$\beta(x)$)
- Pero esto es *exactamente* lo que se supone para el popular modelo de regresión lineal (es decir,$\beta(x) = dy/dx = \beta_1$, una *constante*)

Ejemplo: efectos marginales simulados

```r
plot(ghat,gradients=TRUE,neval=250)
lines(x,-2*pi*sin(2*pi*x),col=2)
abline(h=coef(ghat.ols)[2],col=3)
legend("topleft",c("Kernel ME","DGP ME","Linear ME"),col=1:3,lty=1,bty="n")
```



Efectos marginales de datos mixtos

Efectos marginales de datos mixtos

- Considere una función media condicional multivariante$g(x)$ dónde$x$está compuesto por$q$continuo,$r$desordenado, ys predictores ordenados
- Como en el ejemplo univariado anterior, si el$j$el predictor es continuo, es decir,$x_j \in \mathbb{R}$, entonces$\hat{\beta}_j(x)$es la primera función derivada parcial de$\hat{g}(x)$Con respeto a$j$el predictor, es decir,

$$\hat{\beta}_j(x) = \frac{\partial \hat{g}(x)}{\partial x_j} = \frac{\hat{m}^{(j)}(x)}{\hat{f}(x)} - \hat{g}(x)\frac{\hat{f}^{(j)}(x)}{\hat{f}(x)}$$

Efectos marginales de datos mixtos multivariados

- Si el$j$El predictor está desordenado en$\mathcal{D} = \{a, b, c\}$, entonces $\hat{\beta}_j(x)$es la diferencia entre$\hat{g}(x)$cuando$x_j^u = b$versus$\hat{g}(x)$cuando $x_j^u = a$, y entre$\hat{g}(x)$evaluado en$x_j^u = c$contra en$x_j^u = a$($a$es la categoría *base*)

$$\hat{\beta}_j(x) = \hat{g}(x_{(-j)}, x_j = l) - \hat{g}(x_{(-j)}, x_j = a),\ l = b, c$$

- Si el$j$Cuando el predictor está ordenado, tenemos dos opciones, a saber, tomar diferencias como en el caso desordenado, o tomar diferencias entre elementos *sucesivos* del conjunto ordenado (es decir, entre$a$y$b$entonces entre$b$yc)

Regresión multivariada

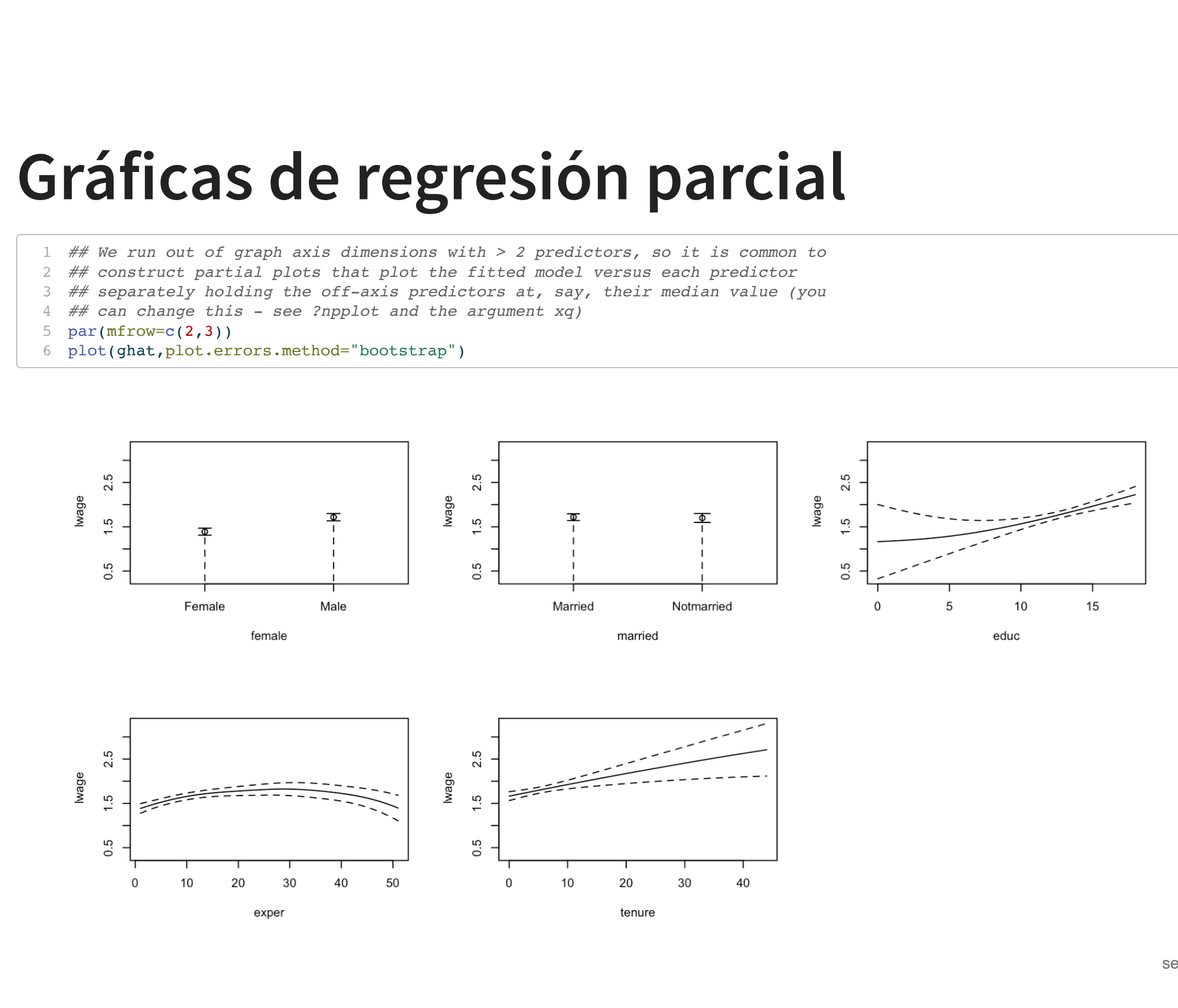
- Usamos `wage1`los datos de Wooldridge que contienen predictores numéricos y categóricos (enlace a la descripción de salario1)
- Hacemos una regresión `lwage`en predictores categóricos `female`y `married`predictores numéricos `educ`, `exper`y`tenure`
- La fórmula para los modelos de regresión es `lwage~female+married+educ+exper+tenure`
- Para el modelo no paramétrico, esto solo enumera los predictores
- Para el modelo paramétrico impone estructura lineal

Regresión no paramétrica multivariada

```r
library(np)
data(wage1)
ghat <- npreg(lwage ~ female + married + educ + exper + tenure, data=wage1, regtype="ll")
summary(ghat)
##
## Regression Data: 526 training points, in 5 variable(s)
##                      female   married     educ    exper   tenure
## Bandwidth(s): 1.058363e-07 0.2881137 5.148991 7.185339 28.00628
##
## Kernel Regression Estimator: Local-Linear
## Bandwidth Type: Fixed
## Residual standard error: 0.36823
## R-squared: 0.520417
##
## Continuous Kernel Type: Second-Order Gaussian
## No. Continuous Explanatory Vars.: 3
##
## Unordered Categorical Kernel Type: Aitchison and Aitken
## No. Unordered Categorical Explanatory Vars.: 2
```

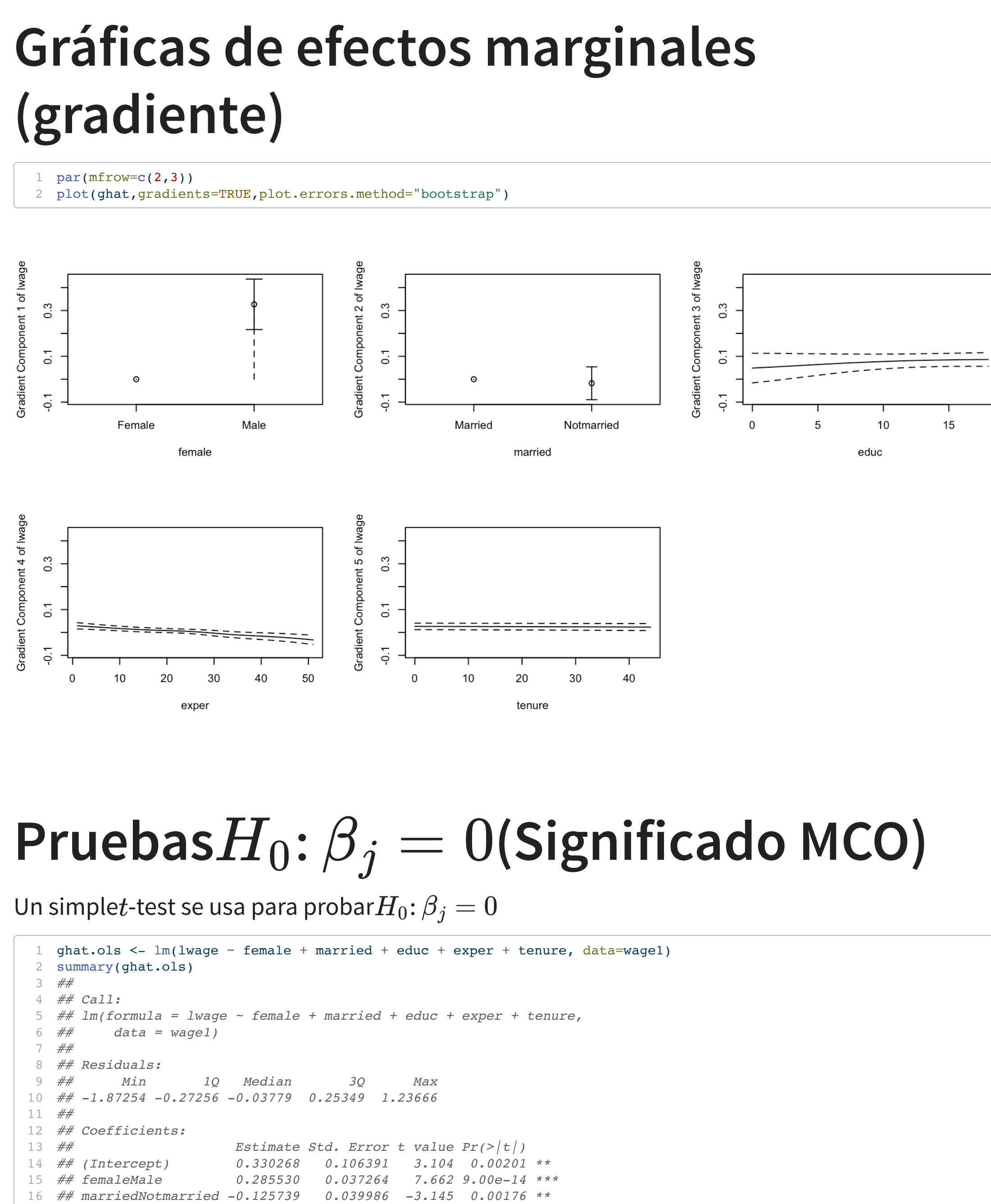
Gráficas de regresión parcial

```r
## We run out of graph axis dimensions with > 2 predictors, so it is common to
## construct partial plots that plot the fitted model versus each predictor
## separately holding the off-axis predictors at, say, their median value (you
## can change this - see ?npplot and the argument xq)
par(mfrow=c(2,3))
plot(ghat,plot.errors.method="bootstrap")
```



Gráficas de efectos marginales (gradiente)

```r
par(mfrow=c(2,3))
plot(ghat,gradients=TRUE,plot.errors.method="bootstrap")
```



PruebasH_0: $\beta_j = 0$(Significado MCO)

Un simple$t$-test se usa para probarH_0: $\beta_j = 0$

```r
ghat.ols <- lm(lwage ~ female + married + educ + exper + tenure, data=wage1)
summary(ghat.ols)
##
## Call:
## lm(formula = lwage ~ female + married + educ + exper + tenure,
##     data = wage1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.87254 -0.27256 -0.03779  0.25349  1.23666
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)        0.330268   0.106391   3.104  0.00201 **
## femaleMale         0.285530   0.037264   7.662 9.00e-14 ***
## marriedNotmarried -0.125739   0.039986  -3.145  0.00176 **
## educ               0.083905   0.006973  12.033  < 2e-16 ***
## exper              0.003134   0.001682   1.863  0.06300 .
## tenure             0.016867   0.002955   5.707 1.93e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4125 on 520 degrees of freedom
## Multiple R-squared:  0.4036, Adjusted R-squared:  0.3979
## F-statistic: 70.38 on 5 and 520 DF,  p-value: < 2.2e-16
```

PruebasH_0: $\beta_j(x) = 0$(Significado)

- Las hipótesis nula y alternativa son

$$\begin{aligned} H_0\colon &\quad \beta_j(x) = 0 \text{ for all } x \text{ (a.e.)} \\ H_A\colon &\quad \beta_j(x) \neq 0 \text{ for some } x \text{ on a set with + measure} \end{aligned}$$

- Una estadística de prueba factible$\hat{\lambda} \geq 0$es dado por

$$\hat{\lambda} = \begin{cases} n^{-1}\sum_{i=1}^{n} \hat{\beta}_j(X_i)^2 & \text{if } x_j \in \mathbb{R} \\ n^{-1}\sum_{i=1}^{n}\sum_{l=1}^{c-1} \hat{\beta}_j(X_i)^2 & \text{if } x_j \in \mathcal{D} \end{cases}$$

- Un procedimiento de arranque proporciona la distribución nula

PruebasH_0: $\beta_j(x) = 0$(Significado)

un arranqueλ-test se usa para probarH_0: $\beta_j(x) = 0\,\forall\,x$(a)

```r
npsigtest(ghat)
##
## Kernel Regression Significance Test
## Type I Test with IID Bootstrap (399 replications, Pivot = TRUE, joint = FALSE)
## Explanatory variables tested for significance:
## female (1), married (2), educ (3), exper (4), tenure (5)
##
##                      female   married     educ    exper   tenure
## Bandwidth(s): 1.058363e-07 0.2881137 5.148991 7.185339 28.00628
##
## Individual Significance Tests
## P Value:
## female  < 2.22e-16 ***
## married 0.0050125 **
## educ    < 2.22e-16 ***
## exper   < 2.22e-16 ***
## tenure  < 2.22e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Contrafactuales

- La función R `fitted(...)`extrae valores ajustados para cada observación de muestra
- Supongamos que queremos valores ajustados para *valores específicos* de los predictores.
- Usamos la función R `predict(...,newdata=...)`donde `newdata`apunta a un marco de datos que contiene *named y cast* Xvalores para los que queremos predicciones
- Generemos un marco de datos llamado que `df`contenga 1 fila para generar el salario logarítmico previsto para hombres solteros con educación media, antigüedad laboral y experiencia laboral.

Contrafactuales

```r
attach(wage1)
df <- data.frame(female = factor("Male", levels=levels(female)),
                 married = factor("Notmarried", levels=levels(married)),
                 educ = median(educ),
                 tenure = median(tenure),
                 exper = median(exper))
head(df)
##   female    married educ tenure exper
## 1   Male Notmarried   12      2  13.5
predict(ghat, newdata=df)
## [1] 1.699066
## Or you could use ghat <- npreg(...,newdata=df) and fitted(ghat)
## ghat <- npreg(lwage ~ female + married + educ + exper + tenure,
##               data=wage1,
##               regtype="ll",
##               newdata=df)
## fitted(ghat)
```

Si `df`tuviera varias filas, obtendríamos una predicción para cada fila (es decir, un vector de predicciones)

Quarto: Documentos técnicos reproducibles

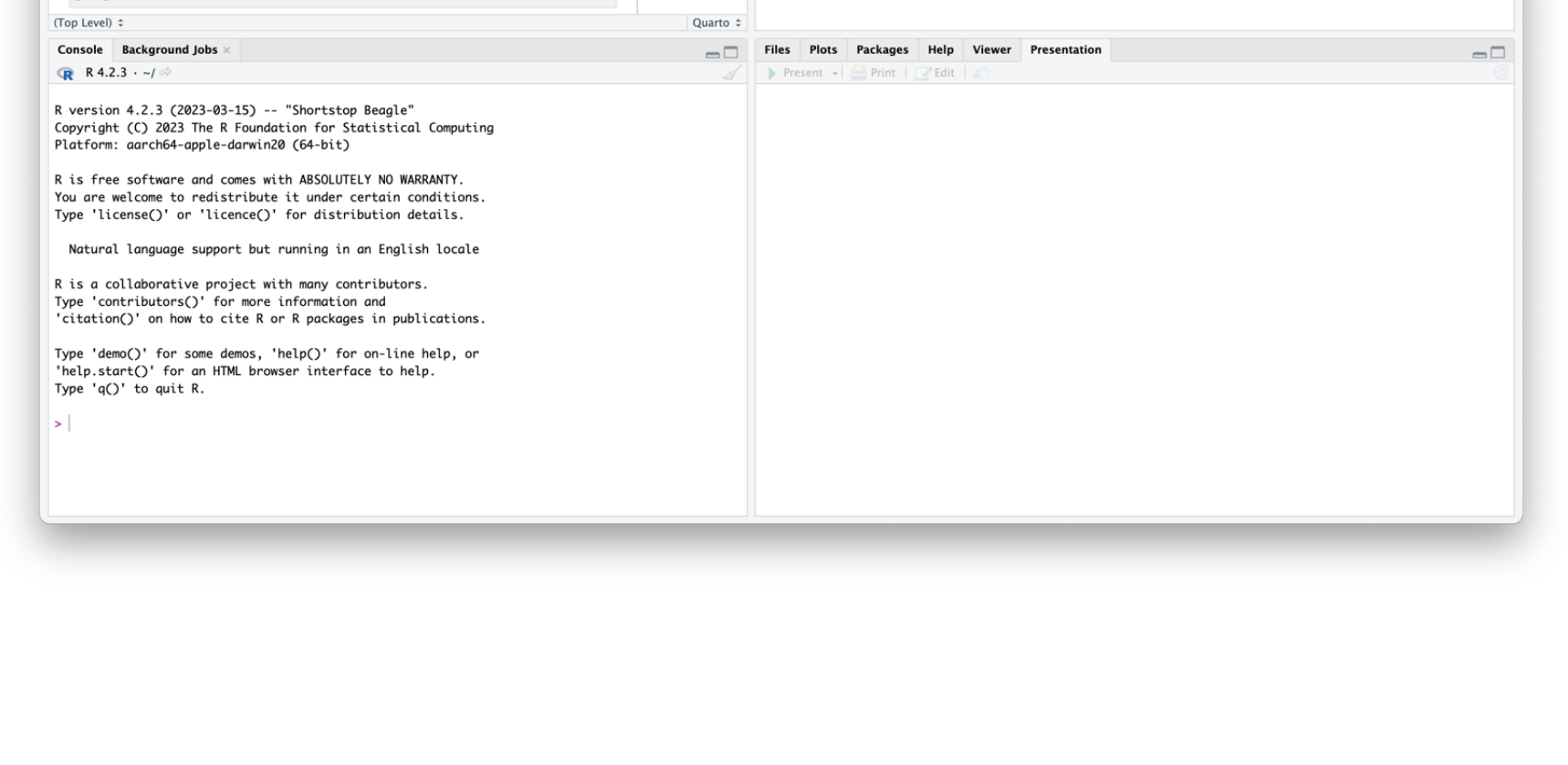
Un flujo de trabajo subóptimo

- La forma antigua de investigar es ejecutar sus análisis usando R, luego copiar manualmente figuras y tablas en un procesador de texto (por ejemplo, MS Word) o una máquina tipográfica (por ejemplo,$\TeX$)
- Así que usted termina manteniendo archivos narrativos, de figuras y códigos separados y siendo su coordinador intermediario.
- ¡Y se ha comprometido previamente con un formato de salida!
- Luego encuentra un código o un error de datos y necesita rehacer figuras, tablas, etc., o necesita cambiar los formatos de salida
- Este es un proceso brutalmente ineficiente y propenso a errores porque requiere un intermediario (es decir, ¡usted!)

Un flujo de trabajo optimizado

- Quarto coloca su código y narrativa en el mismo archivo
- Liberados de las funciones de coordinación, surgen nuevas posibilidades
  - ahora se refiere *automáticamente* a los resultados analíticos
  - ahora describe *condicionalmente* los resultados analíticos
- Si se actualiza el código o los datos, las figuras y tablas se actualizan automáticamente en su documento de salida
- Quizás lo mejor de todo es que su archivo Quarto se puede *representar* en cualquier formato de salida imaginable sin problemas a través de Pandoc
- (Quarto , GitHub y renv son compatibles con RStudio)

Interfaz del editor RStudio Quarto



```
---
title: "Untitled"
format: html
editor: visual
---
```

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see https://quarto.org.

Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
{r}
1 + 1
```

```
R version 4.2.3 (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: aarch64-apple-darwin20 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```

Resumen

- Apenas hemos arañado la superficie y nos hemos saltado detalles técnicos.
- Sin embargo, ahora tiene cierta perspectiva y tiene acceso a herramientas modernas.
- Puede consultar Racine (2019) si desea profundizar más (pruebas detalladas, código R, ejemplos, etc.)
- Muchos de los enlaces en estas diapositivas también pueden ser útiles.
- Puede ponerse en contacto conmigo en racinej@mcmaster.ca
- ¡Ha sido un placer guiarte! ¡Adiós!

Referencias

Hayfield, Tristen y Jeffrey S. Racine. 2008. "Econometría no paramétrica: el paquete np ". *Revista de software estadístico* 27 (5): 1–32. https://doi.org/10.18637/jss.v027.i05 .

Racine, Jeffrey S. 2012. "RStudio: un IDE independiente de la plataforma para R y Sweave ". *Revista de Econometría Aplicada* 27 (1): 167–72. http://www.jstor.org/stable/41337225 .

———. 2019. *Una introducción a la teoría y práctica avanzadas de la econometría no paramétrica: un enfoque replicable usando R* . Prensa de la Universidad de Cambridge. https://doi.org/10.1017/9781108649841 .